

Diagnosing and Recovering MemGPT’s Function-Calling Failures in Small Open-Source LLMs via Knowledge Distillation

Hyogi Kim

Department of Computer Science & Engineering, UNIST

jeff4380@unist.ac.kr

1 Research paper summary

- **Title:** MemGPT: Towards LLMs as Operating Systems
- **Authors:** Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, Joseph E. Gonzalez (UC Berkeley)
- **Venue & Year:** CoRR abs/2310.08560 (2023); v2 Feb 2024
- **URL:** <https://arxiv.org/abs/2310.08560>

Background. Transformer-based LLMs are the backbone of modern conversational AI, yet their fixed-length context windows fundamentally limit their applicability in two practical regimes: *multi-session dialogue*, where a virtual assistant must recall facts across weeks or months of interaction, and *long-document analysis*, where legal or financial documents easily exceed million-token scales. The naive remedy of extending the context window is unattractive on two grounds. First, the quadratic cost of self-attention makes pretraining longer-context models prohibitively expensive. Second, even when long-context models exist, prior work (Liu et al., 2024) shows that they fail to attend uniformly across the window—the “lost in the middle” effect. The authors therefore argue that the right abstraction is not a larger context but a *managed* one, and look to a four-decade-old precedent for inspiration: the virtual memory paging system of traditional operating systems, which gives applications the illusion of memory far larger than physical RAM by paging data to and from disk on demand.

Summary of contributions. The paper makes four connected contributions. **(1) An OS-inspired memory hierarchy.** MemGPT partitions storage into *main context* (the LLM’s prompt tokens; analogous to RAM) and *external context* (vector-indexed databases; analogous to disk). Main context is further divided into a read-only system prompt, a small writable *working context* for persistent facts, and a FIFO queue holding the rolling message history. External context comprises *recall storage* (full conversation log, searchable by embedding similarity) and *archival storage* (an unbounded read/write knowledge store). **(2) Self-directed memory management via function calls.** Rather than offloading memory decisions to a separate orchestrator, the LLM itself decides when to write (`working_context.append`, `archival_storage.insert`) and when to read (`recall_storage.search`, `archival_storage.search`) through structured function calls. A queue manager monitors token usage and issues a “memory pressure” system alert when the FIFO queue nears the context limit, prompting the model to commit important content to external storage before older messages are flushed and recursively summarized. **(3) Event-driven control flow with function chaining.** Each function call can set `request_heartbeat=true`, which immediately re-invokes the LLM with the function’s output appended to context. This enables multi-hop retrieval—paging through results, chaining lookups, performing nested key dereferencing—without user intervention. **(4) New evaluation tasks.** The authors extend the Multi-Session Chat (MSC) dataset with a *Deep Memory Retrieval* (DMR) task that probes whether an agent remembers content from prior sessions, and introduce a *nested key-value retrieval* task that tests multi-hop reasoning over external storage. On DMR, MemGPT with GPT-4 Turbo reaches **93.4% accuracy versus 35.3%** for the recursive-summarization baseline; on multi-document QA over a 20M-passage Wikipedia index, MemGPT’s accuracy stays essentially flat as

the retrieved-document count grows, while fixed-context baselines degrade once truncation kicks in; on the nested KV task, MemGPT is the only configuration that maintains nonzero accuracy beyond two nesting levels.

Limitations and discussion. Three limitations deserve attention. First, the approach is heavily dependent on a strong function-calling model: MemGPT with GPT-3.5 underperforms MemGPT with GPT-4 by a wide margin on every task, suggesting the memory abstraction transfers poorly to smaller or open-source models without dedicated fine-tuning. Second, the system prompt encoding the function schema and memory-management instructions itself consumes a nontrivial slice of the context window, eating into the budget for actual content. Third, the authors note a curious failure mode in document-QA: the agent often *stops paging* through retrieved results before exhausting them, even when the gold passage lies deeper in the ranking. This points to an unresolved exploration–exploitation tension—the LLM has no principled stopping rule for when to keep searching versus when to answer. The paper also evaluates only on English-language tasks, leaves the working-context write policy entirely to LLM discretion (with no analysis of write consistency across runs), and does not benchmark against contemporaneous memory-augmented baselines such as MemoryBank (Zhong et al., 2024). Despite these caveats, the results are convincing: gains over recursive summarization are large and consistent, and the design is reproducible—the released code became the Letta framework, now a standard reference implementation in the field.

Why this paper. I chose MemGPT because it sits at the intersection of two threads I find genuinely compelling: the practical, deployment-relevant question of how commercial assistants like ChatGPT and Claude actually remember users across sessions, and the more theoretical question of how to avoid the context-window bottleneck without simply scaling parameters. Most memory papers I had read previously treated memory as a passive store queried by an external retriever; MemGPT reverses this—the LLM itself becomes the memory manager, which struck me as conceptually cleaner and one with strong industrial uptake (Anthropic’s API Memory Tool, released September 2025, mirrors MemGPT’s explicit-tool-call design almost directly). After a close reading, I do feel I gained what I hoped for: a concrete architectural vocabulary (working context, recall, archival, heartbeat) that I can now use to compare other Memory-Augmented RAG systems on common terms.

Wider research context. MemGPT sits in the lineage of Retrieval-Augmented Generation (Lewis et al., 2020; Karpukhin et al., 2020), but generalizes RAG along two axes. First, it replaces a fixed retriever-reader pipeline with a self-directed read/write loop, in the spirit of FLARE (Jiang et al., 2023) where the model decides *when* retrieval is necessary. Second, it shifts the retrieval target from a static document corpus to a dynamic, agent-written memory store—a move that directly enables the multi-session chat behavior commercial assistants now ship as a product feature. More broadly, the paper exemplifies a recurring pattern in NLP: when scaling a representation hits a wall (longer contexts, more parameters), the productive next move is often architectural rather than parametric. The OS analogy is a particularly clean instance, and the same hierarchical-memory idea has since been extended in MemoryBank, Mem0, A-MEM, and MemOS. For the broader course themes, MemGPT illustrates how language representation problems—what to keep in working memory, what to forget, how to compress evicted content—map onto classical systems problems, and how function-calling LLMs blur the line between model and program.

2 Project description

Goal. This project investigates a critical but underexplored limitation of MemGPT: its self-directed memory management via function calling degrades severely when the underlying LLM is replaced with a smaller or open-source model. As shown in Table 2 of the paper, replacing GPT-4 Turbo with GPT-3.5 Turbo drops DMR accuracy from 93.4% to 66.9% under the identical MemGPT architecture—a 26.5 point gap that the authors attribute to “limited function calling capabilities” without further analysis. My project asks two connected scientific questions. **(1) Diagnosis:** Is this failure caused by a lack of domain knowledge, or by a lack of the behavioral pattern needed to manage memory correctly? I hypothesize that the bottleneck is the latter—smaller models possess the relevant knowledge but fail to invoke the right function at the right time. **(2) Recovery:** Can knowledge distillation from GPT-4, applied via fine-tuning on GPT-4’s MemGPT execution trajectories, recover the lost performance? I will compare full fine-tuning (FT) and parameter-efficient LoRA fine-tuning to assess the cost-performance tradeoff. The project is tightly coupled to the chosen paper: it takes MemGPT’s architecture as fixed and intervenes only at the model level, making the findings directly comparable to the paper’s published numbers.

Task. I adopt the same two evaluation domains as the original paper without modification. **(1) Multi-session chat:** The Deep Memory Retrieval (DMR) task from the extended MSC dataset, where the agent must answer a question about a topic discussed in a prior session. **Input:** five prior conversation sessions plus a probing question from session 6. **Output:** a free-form answer that requires retrieving and reasoning over prior-session content. **(2) Document analysis:** Multi-document QA over the NaturalQuestions-Open subset, where the agent retrieves from a 20M-passage Wikipedia index stored in archival memory. **Input:** a factoid question. **Output:** a short answer with the supporting passage. Keeping the tasks identical to the paper is intentional: it allows my results to be directly placed alongside the paper’s Table 2 and Figure 5.

Data. I will use the two public datasets released with the MemGPT paper. (1) The extended MSC corpus with DMR question/answer pairs (5 sessions $\times$ $\sim$ 12 messages plus a 6th probing session). (2) The 50-question NaturalQuestions-Open subset with the precomputed 20M-passage Wikipedia embedding index (PostgreSQL/pgvector with HNSW indexing). In addition, I will collect **GPT-4 Turbo MemGPT execution trajectories**—sequences of (context, function call, function output) triples—by running GPT-4 Turbo + Letta on the training split of the MSC corpus. These trajectories serve as both the pseudo ground truth for the diagnostic analysis and the supervision signal for fine-tuning. I estimate roughly 2,000–3,000 trajectory steps are needed, which is feasible within API budget constraints.

Methods. The project proceeds in two stages.

Stage 1: Diagnostic Analysis (Oracle Experiment). To test whether small-model failure is caused by knowledge deficit or behavioral pattern deficit, I will run an oracle experiment. For each test instance, I inject GPT-4’s ground-truth function call sequence directly into the small model’s context, forcing it to receive the correct retrieved content, and ask it only to generate the final answer. If the small model’s accuracy recovers to near GPT-4 levels under oracle function calls, this confirms that the bottleneck is behavioral (the model knows the answer but cannot navigate to it). If accuracy remains low, knowledge deficit is implicated. I will further categorize observed failures in the non-oracle setting into three types: **Type 1 (Retrieval Miss)**—the small model answers without searching when GPT-4 would have searched; **Type 2 (Retrieval Hallucination)**—the small model searches unnecessarily or with a malformed query; **Type 3 (Chain Failure)**—the small model stops function chaining prematurely where GPT-4 would have issued a follow-up call via `request_heartbeat=true`. The distribution of failure types will directly motivate the composition of the fine-tuning data in Stage 2.

Stage 2: Knowledge Distillation via Fine-Tuning. Using the collected GPT-4 trajectories as supervision, I will fine-tune two variants of a small open-source model (Llama-3-8B or Mistral-7B): **(a) Full Fine-Tuning (FT)**—standard supervised fine-tuning on the full trajectory data, treating each (context, function call) pair as a next-token prediction target; **(b) LoRA Fine-Tuning**—the same objective but with low-rank adaptation (rank $r \in \{8, 16\}$, $\alpha = 32$) applied to the attention layers only, keeping the base model frozen. The training data will be stratified by failure type identified in Stage 1, so that underrepresented failure patterns (e.g., Chain Failure) are upsampled proportionally. Both the base Letta framework and the LoRA implementation (via HuggingFace PEFT) are downloaded; the trajectory collection pipeline and the failure-type classifier are implemented by me.

Baselines. I will compare all systems on the same evaluation protocol. **(1) GPT-4 Turbo + vanilla MemGPT**—the paper’s strongest result, serving as the performance ceiling. **(2) Small model + vanilla MemGPT**—the unmodified small model under the original MemGPT prompt, establishing how much performance is lost without any intervention. **(3) Small model + Full FT**—distillation without parameter efficiency constraints. **(4) Small model + LoRA FT**—the parameter-efficient variant. Baselines (1) and (2) are directly comparable to the paper’s published Table 2 numbers; baselines (3) and (4) are implemented by me.

Evaluation. **(1) DMR accuracy and ROUGE-L recall**, matching the paper’s Table 2 protocol (LLM-as-judge for accuracy; ROUGE-L recall to account for verbosity). Published reference: GPT-4 Turbo + MemGPT achieves 93.4% / 0.827; GPT-3.5 + MemGPT achieves 66.9% / 0.629. My target is for the small model + LoRA FT to close at least half the gap to GPT-4 (i.e., $\geq 80\%$ accuracy). **(2) Document QA accuracy**, matching Figure 5, evaluated as a function of retrieved document count K . **(3) Failure type distribution**, reported as the fraction of test instances falling into Type 1/2/3 for each system. This diagnostic metric—absent from the original paper—is my primary analytical contribution: it shows not just *how much* each method improves but *which behavioral failures* it resolves. **(4) Parameter and compute efficiency**, comparing FT vs. LoRA in terms of trainable parameters and GPU hours, to evaluate the practical tradeoff between performance recovery and resource cost.

References. Hu et al. (2022) LoRA: Low-Rank Adaptation of Large Language Models, ICLR. Jiang et al. (2023) Active Retrieval Augmented Generation, EMNLP. Karpukhin et al. (2020) Dense Passage Retrieval for Open-Domain QA, EMNLP. Lewis et al. (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS. Liu et al. (2024) Lost in the Middle: How Language Models Use Long Contexts, TACL. Packer et al. (2024) MemGPT: Towards LLMs as Operating Systems, CoRR abs/2310.08560. Xu et al. (2022) Beyond Goldfish Memory: Long-Term Open-Domain Conversation, ACL. Zhong et al. (2024) MemoryBank: Enhancing LLMs with Long-Term Memory, AAAI.